35

BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık İşlemler: 2-Phase Commit (2PC) vs Saga Pattern (Orchestration vs Choreography)

Distributed Transactions: 2-Phase Commit (2PC) vs Saga Pattern

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ 1. TWO-PHASE COMMIT (2PC) - KATI KİLİT, YAVAŞ, BLOKLAYAN ]
Koordinatör ──1. Hazır mısın? (Prepare)──► [DB 1] (Kilitler ve bekler!)
Koordinatör ──1. Hazır mısın? (Prepare)──► [DB 2] (Kilitler ve bekler!)
Koordinatör ──2. COMMIT / ROLLBACK ──────► İkisi de aynı anda onaylar!

[ 2. SAGA PATTERN (TELAFİ EDİCİ İŞLEMLER / COMPENSATING TRANSACTIONS) ]
Adım 1: Para Çekildi ($100) ──► Başarılı
Adım 2: Stok Rezerve Edildi ──► Başarılı
Adım 3: Kargo Oluşturuldu   ──► HATA ÇIKTI! (Kargo Şirketi Çöktü)
Geri Alma (Compensating):
Adım 2-Ters: Stok Rezervasyonunu İPTAL ET
Adım 1-Ters: Parayı Kullanıcıya İADE ET ($100)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

3 arkadaşın birlikte tatile gitmesini düşünün: 1) İki Fazlı Onay (2PC): 'Üçümüz de aynı anda uçak biletini alıyoruz, 3'e kadar sayacağım: 1-2-3 al!' dersiniz. Biri son anda 'İnternetim koptu' derse diğer ikisi de bileti alamaz, herkes donar kalır! 2) Saga Deseni: Herkes kendi işini sırayla yapar: Ali bileti alır, Veli oteli tutar, Can arabayı kiralar. Can 'Araba kalmamış' derse ne olur? Veli hemen oteli ücretsiz iptal eder, Ali de uçak biletini iade eder (Telafi İşlemi - Compensating Transaction)! Herkes kendi telafi planını çalıştırarak sistemi eski haline döndürür!

🇬🇧 ENGLISH (ELI5)

Booking a vacation across 3 independent systems: 1) Two-Phase Commit (2PC): The coordinator freezes all databases: 'I am counting to 3; everyone lock resources and prepare to commit simultaneously!' If airline booking stalls, hotel and car rental databases sit locked hostage! 2) Saga Pattern: Decoupled local transactions with compensatory rollbacks: Step 1 charges the credit card. Step 2 reserves the flight. Step 3 reserves the hotel. If hotel reservation fails, compensate backwards: Step 2-Compensate refunds the flight, Step 1-Compensate refunds the card! No long-lived database locks!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) Two-Phase Commit (2PC / XA): Güçlü ACID garantisi verir. Koordinatör önce 'Prepare' aşamasında tüm düğümlerden 'Hazırım' sözü alır, ardından 'Commit' gönderir. Büyük kusurları: Koordinatör çökerse kilitler sonsuza kadar asılı kalır (Blocking); ağ gecikmeleri nedeniyle throughput berbattır; mikroservislerde ASLA ölçeklenmez. 2) Saga Pattern: Uzun süren dağıtık işlemleri yerel ACID transaction'lar zincirine böler. Bir adım başarısız olursa, önceki başarılı adımlar 'Compensating Transactions' (Telafi Edici İşlemler) ile geri alınır. İki uygulama biçimi: A) Choreography: Olay tabanlıdır (Event-Driven); servisler Kafka olaylarını dinleyip birbirini tetikler. B) Orchestration: Merkezi bir 'Saga Orchestrator' (Devlet Makinesi - State Machine) tüm adımları yönetir ve hata anında telafi adımlarını sırayla tetikler (Büyük sistemlerde takibi kolay olduğu için önerilir).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Two-Phase Commit (2PC): Provides strict linearizable ACID across databases. Coordinator executes Phase 1 (Prepare) where participants acquire locks, then Phase 2 (Commit). Flaws: Synchronously blocks threads and database locks; if the coordinator crashes mid-flight, participants deadlock indefinitely. Strictly unviable for high-scale microservices. 2) Saga Pattern: Decomposes distributed transactions into sequences of isolated local transactions. If a local transaction fails, compensating transactions execute in reverse order to undo side-effects. Execution models: A) Choreography: Decentralized event pub/sub across Kafka topics; difficult to trace at scale. B) Orchestration: A centralized state machine orchestrator coordinates step execution and rollback compensation trees; universally preferred in enterprise architectures for auditability.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```java
// Java 25: Saga Orchestrator (Telafi Edici İşlem Motoru)
import java.util.*;

public class SagaOrchestrator {
    public interface SagaStep {
        boolean execute();
        void compensate(); // Telafi edici geri alma
    }

    public static boolean executeSaga(List<SagaStep> steps) {
        Deque<SagaStep> executedSteps = new ArrayDeque<>();
        for (SagaStep step : steps) {
            if (step.execute()) {
                executedSteps.push(step);
            } else {
                IO.println("⚠ HATA OLUŞTU! Geri sarma (Compensating) başlatılıyor...");
                while (!executedSteps.isEmpty()) {
                    executedSteps.pop().compensate();
                }
                return false;
            }
        }
        return true;
    }
}

void main() {
    var paymentStep = new SagaOrchestrator.SagaStep() {
        public boolean execute() { IO.println("1. Ödeme Alındı: $100"); return true; }
        public void compensate() { IO.println("1-TELAFİ: Kullanıcıya $100 İade Edildi!"); }
    };

    var inventoryStep = new SagaOrchestrator.SagaStep() {
        public boolean execute() { IO.println("2. Stok Rezerve Edilemedi! (HATA)"); return false; }
        public void compensate() { IO.println("2-TELAFİ: Stok Serbest Bırakıldı."); }
    };

    boolean success = SagaOrchestrator.executeSaga(List.of(paymentStep, inventoryStep));
    IO.println("Saga Başarılı mı? " + success);
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta mikroservisler arasında veri tutarlılığı sorulduğunda 'Two-Phase Commit (2PC) kurarım' demeyin! 2PC dağıtık sistemlerde performans katili olarak bilinir. Mülakatçıya doğrudan Saga Deseni (Orchestration tabanlı) ve telafi edici işlemleri anlatın.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch 2PC for modern distributed cloud microservices. 2PC causes crippling latency and lock contention. Instantly propose the Saga Pattern with explicit Compensating Transactions.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- 2PC kilitleri asılı tuttuğu için mikroservis mimarilerinde ölçeklenemez.
- Saga Deseni işlemi yerel parçalara böler ve hata anında ters telafi adımlarını çalıştırır.
- Orchestrator tabanlı Saga mimarisi, karmaşık akışlarda gözlemlenebilirlik ve yönetim kolaylığı sağlar.

🎯 Key Takeaways for the Interview

- 2PC introduces synchronous lock blocking that cripples microservice throughput.
- The Saga Pattern executes local transactions with reverse compensating rollback hooks.
- Orchestrator-based Sagas provide superior visibility, state tracking, and auditing.